

NAME:	ENTRY NO:
-------	-----------

SIL765 & SIL7165: Networks & System Security

Major Exam - May 4, 2026

Time: 120 minutes

Maximum Marks: 100

Number of Questions: 10

Instructions:

1. This is a closed-mode examination, i.e., you must not have any study material while taking the exam. All electronic devices must be stored away from you during the exam.
2. Please attempt all questions. If you feel any question/statement is ambiguous, please write your assumptions clearly, then answer as per your assumptions.
3. You must answer the question in the box assigned for each question. Any text outside the given box will not be evaluated.
4. Please read the honour code given below carefully, and sign in the designated area to accept the same. Your answer sheet may not be evaluated in the absence of such a signature.

Honour Code: As a student of IIT-D, I will not give or receive aid in examinations. I will do my share and take an active part in seeing to it that others, as well as myself, uphold the spirit and letter of the Honour Code.

Your Signature

Question 1 [10 marks]

A company trains a malware classifier using sensitive application samples collected from enterprise customers. The trained model is deployed through an API that allows external queries. An attacker can issue a large number of queries to the model and may later obtain access to the deployed model parameters.

- (a) [2 marks] Identify and explain **two distinct privacy attacks** that are possible in this setting. Clearly describe what information the attacker can infer in each case.

Solution:**Membership inference:**

The attacker tries to decide whether a particular sample x was present in the training set:

$$x \in D_{\text{train}} ?$$

If the model is overconfident on training examples, the attacker may infer that a particular customer's malware sample was used during training.

Model inversion:

The attacker tries to reconstruct sensitive properties of the training data. For example, by querying the classifier repeatedly, the attacker may infer characteristic features of malware samples belonging to a particular organisation or family.

Thus, membership inference reveals whether a record was used during training, while model inversion reveals information about the contents or features of training records.

- (b) [5 marks] Design a privacy-preserving training framework using a **teacher ensemble** and a **student model**. Explain how **noisy aggregation** and student training reduce privacy leakage.

Solution: A PATE-style privacy-preserving design works as follows.

First, split the sensitive training data into disjoint partitions:

$$D = D_1 \cup D_2 \cup \dots \cup D_n$$

Train one teacher model on each partition:

$$T_i \leftarrow \text{Train}(D_i)$$

For a public unlabelled sample x , each teacher votes:

$$v_i = T_i(x)$$

For each class j , count the number of teacher votes:

$$c_j = |\{i : T_i(x) = j\}|$$

Add noise to the vote count:

$$\tilde{c}_j = c_j + \text{Lap}(\lambda)$$

Release only the noisy aggregated label:

$$\hat{y} = \arg \max_j \tilde{c}_j$$

These noisy labels are then used to train a student model on public or non-sensitive data.

Privacy reasoning: If most teachers agree strongly, the output label does not depend heavily on any single partition, so privacy leakage is small. If the votes are close, the

decision may reveal information about a small number of partitions, so adding noise is essential. The final deployed student model is trained only on public data labelled by the noisy teacher aggregation, so the adversary does not directly inspect the teacher models trained on sensitive customer data.

Thus, noisy aggregation limits leakage from individual training partitions, and student training prevents direct exposure of the sensitive teacher models.

- (c) [3 marks] A sophisticated Advanced Persistent Threat (APT) group has infiltrated a corporate network using a zero-day exploit in a VPN appliance. The attackers established persistence, moved laterally, and exfiltrated sensitive data over encrypted channels.

- i. How would you conduct a forensic investigation to trace the attack's origin, lateral movement, and data exfiltration?

Solution:

Origin tracing: Analyse VPN appliance logs, authentication logs, and exploit artefacts to identify the initial compromise point (e.g., unusual login patterns or zero-day indicators).

Lateral movement: Correlate endpoint logs (EDR), Active Directory logs, and network flow data to track privilege escalation and movement across hosts.

Data exfiltration: Inspect network traffic metadata (NetFlow, proxy logs, DNS logs) to identify large or unusual encrypted outbound connections, even when payload contents are not visible.

- ii. What security controls (preventive, detective, and corrective) would you recommend to prevent future breaches of this nature?

Solution:

Preventive: Patch management, network segmentation, least-privilege access control, and multi-factor authentication (MFA) for remote VPN access.

Detective: Deploy SIEM, EDR, and anomaly-based intrusion detection systems (IDS) to detect unusual behaviour, privilege escalation, and lateral movement.

Corrective: Maintain an incident response plan, isolate compromised systems, rotate credentials, and restore systems from clean backups after containment.

Question 2 [10 marks]

A user visits multiple websites using a standard browser. Each website includes third-party scripts (e.g., analytics and ads) that send requests to third-party domains. The browser automatically attaches:

- Cookies associated with the third-party
- HTTP headers (e.g., **Referer**, **User-Agent**)

Additionally, JavaScript can collect browser fingerprint attributes.

- (a) [2 marks] Formally explain how a third-party tracker can link a user's activity across multiple websites. Your answer must include:

- A precise description of the information available to the tracker, and
- A formal representation of how identity linking is achieved.

Solution: The third-party requests include:

- Cookie: $ID = c$

- Referrer: `siteA.com/page1`
- Headers: `User-Agent`, IP address, etc.

Formally, each request observed by tracker T is:

$$r_i = (c, ref_i, h_i)$$

where:

- c = persistent identifier (cookie),
- ref_i = visited site or page,
- h_i = auxiliary fingerprint information (headers and browser attributes).

The tracker builds a user profile:

$$U_c = \{ref_1, ref_2, \dots, ref_n\}$$

Thus, visits across different websites are linked because:

$$r_i.c = r_j.c \Rightarrow \text{same user}$$

Even if cookies are blocked, fingerprinting can still work:

$$f = \text{hash}(h_1, h_2, \dots, h_k)$$

If:

$$f_i \approx f_j$$

then:

$$\text{link}(i, j) = \text{true}$$

Hence tracking persists through:

- Cookies (explicit identity), and
- Fingerprinting (implicit identity).

(b) [6 marks] Design a privacy-preserving browser architecture that prevents cross-site tracking while preserving web functionality. Your answer must include:

- At least **two** protocol-level or browser-level mechanisms, and
- A formal argument showing why cross-site identity linking fails.

Solution: A strong privacy-preserving browser design includes the following mechanisms:

1. Partitioned Storage (First-Party Isolation) [Browser level]

Cookies become partitioned per site and tracker:

$$C_{(site, tracker)}$$

Thus:

$$c_{(siteA,T)} \neq c_{(siteB,T)}$$

Therefore:

$$r_{A.c} \neq r_{B.c} \Rightarrow \text{no linking}$$

2. Referer / Metadata Minimisation [Protocol level]

Restrict the referer field to only the origin:

$$ref = origin(site)$$

instead of exposing the full URL.

This prevents sensitive leakage such as:

`https://site.com/medical?disease=HIV`

3. Fingerprint Randomisation [Browser level]

Add controlled noise to browser fingerprint attributes:

$$h' = h + \epsilon$$

so that:

$$f_i \not\approx f_j$$

making stable fingerprint-based linking unreliable.

4. Blocking Third-Party JavaScript Execution [Browser level]

Restrict or block third-party scripts entirely, preventing browser fingerprint collection.

Why it works:

The tracker now observes:

$$r_i = (c_{(site,T)}, ref', h')$$

Thus:

$$r_A \neq r_B$$

Therefore, cross-site identity linkage fails.

- (c) [2 marks] Consider the AES-128 algorithm, which operates on a 128-bit block using a 128-bit key. Given, the AES S-box is constructed using multiplicative inversion in $GF(2^8)$ (with irreducible polynomial $m(x) = x^8 + x^4 + x^3 + x + 1$) followed by an affine transformation. Explain how the Key Expansion's use of $GF(2^8)$ arithmetic contributes to AES's resistance against related-key attacks.

Solution: In AES key expansion, $GF(2^8)$ arithmetic is used inside the S-box through multiplicative inversion during the SubWord operation. This introduces strong nonlinearity into the generation of round keys.

As a result, even small and structured differences between related keys (e.g., K and K')

are transformed into complex, nonlinearly related round keys rather than preserving simple algebraic relationships. Furthermore, the combination of $GF(2^8)$ operations with RotWord and round constants ensures rapid diffusion of key differences across all generated round keys.

Therefore, attackers cannot easily exploit predictable relationships between related keys, which strengthens AES against related-key attacks.

Question 3 [4 marks]

A company maintains a shared file system. Initially, it uses Access Control Lists (ACLs) for each file. Employees frequently collaborate, and files are often shared dynamically between processes running on behalf of different users. Over time, managing permissions becomes complex and error-prone. To address this, the system architects consider switching to a capability-based system, where processes hold tokens granting access to resources and can pass them to other processes.

- (a) [2 marks] Explain why ACLs become difficult to manage in this scenario. How does the capability-based approach simplify dynamic sharing?

Solution: ACLs become difficult to manage because each file must maintain a list of all users and permissions allowed to access it. In an environment with frequent collaboration and dynamic sharing, permissions must be updated often, making the system complex and error-prone.

In a capability-based system, a process directly holds capabilities (tokens) granting access to resources. Sharing access becomes simpler because a process can transfer a capability to another process without modifying the resource's global permission list.

- (b) [2 marks] In the capability-based system, processes can transfer capabilities to others. Explain one security risk of this feature and how systems ensure that capabilities are not misused or forged.

Solution: One security risk is that a process may unintentionally or maliciously transfer a capability to an unauthorized process, causing privilege propagation or unauthorized access.

Systems prevent misuse or forgery of capabilities by storing them in protected kernel memory or using cryptographically protected/unforgeable tokens that user processes cannot modify directly.

Question 4 [10 marks]

Dissident X operates from within State A, a highly restrictive network environment. Journalist Y operates overseas. Both parties require absolute anonymity regarding their physical locations and IP addresses.

You are tasked with designing the end-to-end communication architecture allowing X and Y to converse securely. Your design must strictly overcome the following adversarial constraints:

1. **The Egress Constraint:** State A's edge routers perform Deep Packet Inspection (DPI). They immediately drop connections exhibiting unrecognised cryptographic handshakes and maintain a real-time, comprehensive blocklist of all publicly known proxy servers and relay IPs.
2. **The Overlay Constraint:** State A operates a highly funded intelligence apparatus that controls approximately 15% of the bandwidth in any global anonymity overlay network. They are capable of executing long-term timing and correlation observation attacks.
3. **The Forensic Constraint:** It is a certainty that X's physical devices will eventually be seized and forensically imaged by State A. State A will extract all long-term private keys and encrypted message archives, and will attempt to mathematically prove in an international court that X authored the messages.

Design the communication stack. You must explicitly define the mechanisms used at the **Network Egress**, **Routing**, and **Application Cryptography** layers. For each layer, logically prove how your specific architectural choices neutralise the corresponding state constraint.

Solution: Network Egress Layer (Neutralizing DPI and IP Blocks)

Mechanism: Decoy Routing

The dissident initiates a TLS connection to a permitted, non-blocked destination (e.g., Not-Blocked.com). An invisible tag is embedded in the cryptographic handshake. A Telex station at a friendly ISP intercepts this tag and redirects the traffic to the overlay.

Why Tor is INCORRECT for Egress: Handshake Metadata: State A uses Deep Packet Inspection (DPI) to identify the specific cryptographic handshake used by Tor. Because Tor's handshake patterns and packet metadata are publicly known and distinct, the edge routers can identify the traffic as "Tor" and immediately drop the connection.

2. Routing Layer (Neutralizing Bandwidth Control and Timing Attacks)

Mechanism: Hidden Services

Logic: Hidden Services allow communication without disclosing an IP address. To counter the 15

3. Application Cryptography Layer (Neutralizing Forensics)

Mechanism: Off-the-Record (OTR) Protocol.

Logic: OTR provides Deniability by publishing old Message Authentication Code (MAC) keys after the session ends. Because these keys become public, anyone could have forged the message archives after the fact. This ensures State A cannot mathematically prove X was the unique author of the messages in court.

Question 5 [10 marks]

A developer writes the following logging function in a network daemon:

```
void log_error(char *user_input) {
    char buffer[256];
    snprintf(buffer, sizeof(buffer), user_input);
    syslog(LOG_ERR, buffer);
}
```

An attacker sends crafted input to this function.

- (a) [2 marks] Explain what type of vulnerability exists in this code and why it arises, even though `snprintf` is used.

Solution: This code contains a **format string vulnerability**. The attacker-controlled `user_input` is passed directly as the format string to `snprintf`. Although `snprintf` limits the number of bytes written to the buffer and helps prevent buffer overflows, it still interprets format specifiers such as `%x`, `%s`, and `%n`. Therefore, an attacker can read or modify memory using crafted input.

- (b) [2 marks] Assume the attacker supplies input like: `"%08x.%08x.%08x.%08x"`
Explain what kind of information the attacker can obtain.

Solution: The `%08x` format specifiers cause values from the program stack to be printed in hexadecimal form. This can leak sensitive information such as stack contents, memory addresses, return addresses, pointers, or other process memory data.

- (c) [2 marks] How can the information obtained in part (b) help the attacker in further exploitation?

Solution: Leaked memory addresses and stack information can help the attacker bypass security protections such as ASLR (Address Space Layout Randomization) or locate important code and data in memory. This information can then be used to make other attacks, such as buffer overflow or return-oriented programming (ROP) attacks, more reliable.

- (d) [4 marks] Suppose we could deploy a mechanism that ensures IP source addresses always correspond to the actual sender of a packet—in other words, that it is impossible for an attacker to spoof source addresses. For each of the following threats, explain whether (and briefly why) the mechanism would either: (i.) Completely eliminate the threat. (ii.) Eliminate some instances of the threat, but not all of them. (iii.) Has no impact on the threat.

1. Reflected cross-site scripting (XSS) attacks.
2. Kaminsky's DNS cache poisoning attack.
3. DoS "amplification" where the attacker sends traffic to an Internet "broadcast" address.
4. Clickjacking.

Solution:

1. **Reflected XSS attacks:**

(iii) **Has no impact.** Reflected XSS attacks rely on malicious scripts being executed in a victim's browser and do not depend on spoofed IP addresses.

2. **Kaminsky's DNS cache poisoning attack:**

(ii) **Eliminates some instances, but not all.** The attack relies in part on forged DNS responses pretending to come from authoritative servers, so preventing spoofing

would make the attack significantly harder. However, other forms of DNS poisoning may still exist.

3. DoS amplification using broadcast addresses:

(i) **Completely eliminate.** These attacks rely on spoofing the victim's IP address so that amplified replies are sent to the victim. Without IP spoofing, the amplification attack would not work.

4. Clickjacking:

(iii) **Has no impact.** Clickjacking is a user-interface deception attack and is unrelated to spoofed IP packet addresses.

Question 6 [10 marks]

You are analysing a proprietary data-ingestion module. In this context, the user-controlled input is stored in `buf`, and `len` represents the total length of that input.

```

1  int process_sensor_data(unsigned char* buf, int len) {
2      if (len < 5) return -1;
3
4      if (buf[0] != 0xDE || buf[1] != 0xAD || buf[2] != 0xC0 || buf[3] != 0xDE) {
5          return -1;
6      }
7
8      int sensor_count = buf[4];
9      int idx = 5;
10     int critical_threshold = 0;
11
12     while (idx < len && sensor_count > 0) {
13         critical_threshold += buf[idx];
14         idx++;
15         sensor_count--;
16     }
17
18     if (critical_threshold == 0xAA) {
19         abort();
20     }
21     return 0;
22 }
```

- (a) [4 marks] An auditor deploys an automated tool that randomly applies bit-flips and byte-insertions to an initial 10-byte input consisting of all zeroes (**0x00**). The tool runs for 24 hours, generating millions of mutated inputs.

Identify the exact line number where execution will terminate for the overwhelming majority of these mutated inputs. Logically and mathematically deconstruct the structural reason why this specific automated approach fails to progress further. Specify two structurally distinct automated discovery architectures from the literature that can bypass this bottleneck, explaining the internal mechanism each uses to successfully execute past the termination point.

Solution: Line 4/5.

Reason: Black-Box Mutational Fuzzing cannot randomly guess a 32-bit magic constant (0xDEADC0DE) with a 1-in-4.2 billion chance.

Solutions: Generation-based (using a spec) or Grey-box (using instrumentation) fuzzing.

- (b) [3 marks] The auditor switches to a framework that uses compile-time wrappers to track internal edge transitions during execution. They initialise this framework with a 5-byte seed file: [0xDE, 0xAD, 0xC0, 0xDE, 0x00].

Detail the internal triage and feedback loop this framework utilises to eventually generate an input that reaches line **19**. Analyse the operational impact of providing this specific 5-byte seed compared to initialising the framework with a 10-byte all-zero seed. What is the precise effect on the discovery timeline, and how does the framework's internal logic dictate this outcome?

Solution: Feedback Loop: Coverage-based fuzzing tracks edge transitions. New paths are saved and mutated further to "evolve" toward the crash.

Impact: Valid seeds drastically decrease discovery time by bypassing the initial header check immediately.

- (c) [3 marks] A developer argues that automated tools are unnecessary and provides a single manual test case: `buf = [0xDE, 0xAD, 0xC0, 0xDE, 0x01, 0xAA]`. They prove this input achieves 100% statement coverage, as every line of code (including line **19**) is executed.

Evaluate the developer's claim. Is their reliance on 100% statement coverage mathematically sound in guaranteeing the `while` loop (lines **12–16**) is fully robust and free of state-management flaws? Justify your logic. If the developer's methodology is flawed, state the specific test-design heuristic required by the literature to rigorously audit a looping construct like this, and specify the exact values that must be passed into `buf[4]` to satisfy it.

Solution: Evaluation: Incorrect.

Justification: 100

Heuristic: Test zero, one, and two iterations of the loop (Input values: 0, 1, 2).

Question 7 [6 marks]

Most Linux Local Privilege Escalations (LPEs) need a race window or a kernel-specific offset. CVE-2026-31431 is a straight-line logic flaw; it needs neither. The same 732-byte Python script roots every Linux distribution shipped since 2017. The exploit was publicly disclosed on 2026-04-29. Here is a small excerpt from the explanation of the exploit:

Page Cache Pages in the Writable Scatterlist

Note: A scatter list is a list of memory segments that allows data to be stored or transferred across multiple non-contiguous memory locations as if it were contiguous.

AF_ALG is a socket type that exposes the kernel's crypto subsystem to unprivileged userspace. A user can open a socket, bind to any AEAD (Authenticated Encryption with Associated Data) template, and invoke encryption or decryption on arbitrary data. No privileges required.

A core primitive underlying this bug is `splice()`: it transfers data between file descriptors and pipes without copying, passing page cache pages by reference. When a user splices a file into a pipe and then into an AF_ALG socket, the socket's input scatterlist holds direct references to the kernel's cached pages of that file. The pages are not duplicated; the scatterlist entries point at the same physical pages that back every `read()`, `mmap()`, and `execve()` of that file.

For AEAD decryption, the input is `AAD || ciphertext || authentication_tag`, where AAD stands for associated authenticated data. Inside `algif_aead.c`, `recvmsg()` sets up the operation as in-place, meaning the same scatterlist serves as both input and output for the crypto algorithm.

The AAD and ciphertext data are byte-copied from the input scatterlist into the output buffer via `memcpy_sglist`. This is a real copy. The page cache pages are only read. But the authentication tag, the last `authsize` bytes of the input scatterlist, is not copied. The kernel retains the scatterlist entries for the tag and chains them onto the end of the output scatterlist using `sg_chain()`:

Input SGL: AAD || CT || Tag

```

      |      |      ^
      | copy |      | sg_chain (still references page cache pages)
      v      v      |
Output SGL:  AAD  ||  CT  -----+

```

The output scatterlist now has two regions: the user's `recvmsg` buffer (containing the copied AAD and ciphertext), followed by the chained tag pages, which still reference the original page cache pages of the target file. The kernel sets `req->src = req->dst`, both pointing to the head of this combined chain:

```

req->src -----+
      |
      v
req->dst --> [ AAD  ||  CT  ] --> [ Tag (page cache pages) ]
      |                  |      |
      +-- RX buffer ----+  +-- chained from TX SGL -+
      |   (user mem)    |   (file's page cache)

```

- (a) [6 marks] Given our understanding and hints from the above text, can you figure out a way an attacker could use this in an exploit?

Solution: This in-place design is the root cause of the vulnerability. It places page cache pages in a writable scatterlist, separated from the legitimate write region by nothing more than an offset boundary. The design assumes that every AEAD algorithm will confine its writes to the intended destination. Still, the API provides no enforcement and documents it as neither a requirement nor an expectation.

Question 8 [5 marks]

Which of the following must be true for a DDoS attack to succeed? Write in the space below all that apply.

1. The attacker has access to traffic generators — whether a botnet, compromised servers, or a single high-bandwidth source — capable of producing sustained attack traffic.
2. The victim's inbound bandwidth or service resource (CPU, connection slots, memory) is finite and exhaustible.
3. The attacker traffic can be scrubbed upstream or rerouted.
4. For reflection/amplification attacks, publicly reachable third-party reflectors exist that will respond to spoofed requests with a response larger than the request.
5. The victim's software contains an unpatched vulnerability that the attacker targets.
6. The attacker can sustain the attack long enough to outlast the victim's mitigation response time.

Solution: 1,2,6, (4)

Point 4 is true only for a subset of DDoS attacks)

Item 1. Without a traffic source of sufficient scale, no resource is exhausted. The source can be a botnet, rented VPS fleet, or a single 100 Gbps server.

Item 2: The universal necessary condition across both attack types — something finite must run out.

Item 3: This is the exact defence against a DDoS attack.

Item 4 is necessary only for the specific subclass of reflection/amplification attacks, so it should be noted with that qualification rather than listed unconditionally. Items 3 and 5 are either irrelevant or actively contrary to attack success.

Item 5: RFC-compliant HTTP GETs, valid DNS queries, and well-formed UDP packets can all be weaponised. No CVE required.

Item 6: DDoS is inherently a war of attrition. If the victim can deploy a CDN, contact their ISP, or activate scrubbing faster than the attacker can sustain load, the attack fails. Botnets go offline, rented capacity expires, operators notice.

Question 9 [10 marks]

Match the following and write your answer in the box below. Each item on the left may match with more than one item on the right. Marks will only be awarded if all selections are correct.

Learning Paradigm	Target / Use Case
A Supervised Learning	1 Classifying malware samples using a labelled dataset of known malware families
B Unsupervised Learning	2 Grouping network flows into behavioural clusters with no prior knowledge of attack types
C Reinforcement Learning	3 Training an automated penetration testing agent that learns optimal attack sequences through trial and error
D Online / Incremental Learning	4 Adapting an intrusion detection model in real-time as novel attack traffic arrives, without full retraining
E Semi-Supervised Learning	5 Detecting phishing emails using a training set of labelled legitimate and malicious samples
	6 Learning an optimal firewall rule update policy in a dynamic network environment
	7 Identifying anomalous user behaviour when only a small fraction of sessions are labelled as malicious
	8 Detecting zero-day network intrusions by finding statistical outliers in unlabelled traffic

Table 1: Match each learning paradigm (A–E) with all applicable usecases (1–10). Each item on the left matches **one or more** items on the right.

Write your solutions below:

A	1, 5	Both require a fully labelled dataset with ground-truth class assignments. Malware family classification and phishing detection are canonical supervised tasks.
B	2, 8	No labels exist or are used. The model discovers structure — clusters or statistical outliers — purely from the data distribution.
C	3, 6	Both involve an agent taking sequential actions in an environment to maximise a reward. Pentesting and firewall policy optimisation are the two canonical RL security applications.
D	4, (6)	The defining property of online learning is the ability to update models on streaming data without storing or replaying the full training set — an essential property when attack distributions shift continuously. (6 not the best fit, as online learning is about how the model is updated)
E	7	A small labelled set (known malicious sessions) plus a large unlabelled set is the precise regime semi-supervised learning is designed for — common in security, where labelling is expensive.

Solution: A note about point 6:

C targets items where the defining feature is what kind of objective is being optimised (policy under reward)

D targets items where the defining feature is how the model is updated (streaming, no full retraining)

Question 10 [5 marks]

Challenge-response mutual authentication is widely considered stronger than password-based authentication. You have seen examples of these while studying Wi-Fi, LTE, and 5G security. Which of the following attacks can succeed against a challenge-response mutual authentication protocol? Select all that apply.

1. Replay attack using a previously captured challenge-response transcript, if nonces are not bound to session state.
2. Man-in-the-Middle attack where the adversary simultaneously runs two protocol sessions — one with each legitimate party — forwarding challenges and responses between them.
3. Pre-image attack on the hash function used to compute the response, if the hash is collision-resistant but not pre-image resistant.
4. Reflection attack, where the attacker returns the server's own challenge back to it as a response, exploiting symmetric key reuse between both directions.
5. Dictionary attack on the secret, if the challenge and response are both observable and the response is computed as HMAC(secret, challenge) with a low-entropy secret.
6. A brute-force attack is impossible if the challenge space is larger than 2^{64} .

Solution: 1, 2, 4, 5

Item 1: If nonces are predictable or session binding is absent, a captured (challenge, response) pair can be replayed. The server has no way to distinguish a fresh response from a recorded one.

Item 2: The adversary acts as a legitimate client to the server and a legitimate server to the client simultaneously, relaying challenges and responses. Neither party detects the MitM because every message is cryptographically valid.

Item 3: Pre-image resistance is exactly what is needed here and is typically satisfied. Collision resistance alone doesn't create a vulnerability — the attacker would need a pre-image attack (find secret given response and challenge), which is precluded by pre-image resistance.

Item 4: If the same key and MAC function are used for both directions, the attacker challenges the server, receives a valid MAC, and reflects it back as their own response to the server's challenge — if the challenges happen to match or the protocol doesn't separate client/server roles in the MAC input.

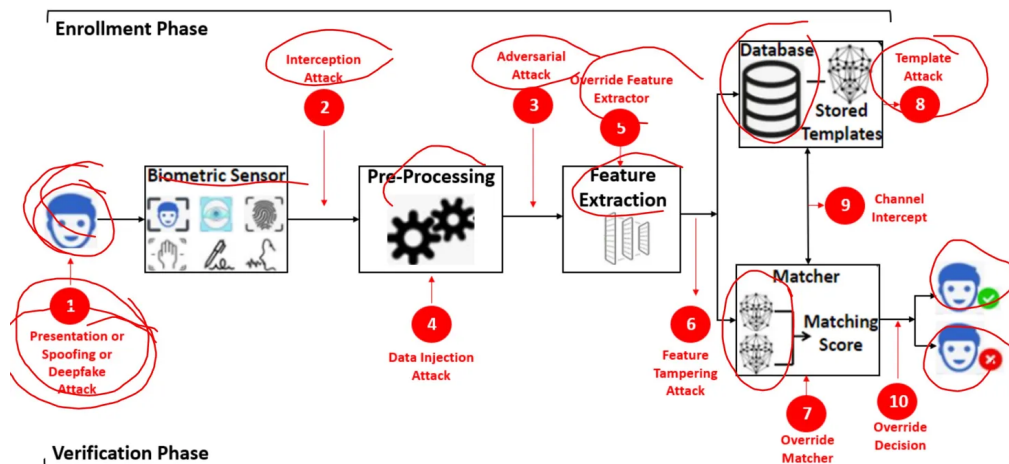
Item 5: The scheme is secure against a computationally bounded attacker with a high-entropy secret. But HMAC is deterministic — given (challenge, response), an attacker can mount an offline dictionary attack by computing HMAC(candidate, challenge) for every candidate until a match is found. No interaction with the server is required after one transcript capture.

Item 6: A large challenge space prevents challenge prediction but says nothing about offline attacks against the secret. The challenge is typically public.

Question 11 [20 marks]

You have been asked to set up a biometrics-based authentication system for your company. You have to start from the beginning of the design and implement the system. Knowing what you do

Attacks Points in a Typical ML System



about ML and its use in security. Match the attack vectors on the left to the property/consequence to the right. Marks will only be awarded if all selections are correct. Each vector may match with more than two items on the right. Write your solutions below:

Attack Vector	Property / Consequence
A Presentation / Spoofing / Deepfake Attack	1 Requires no interaction with the physical biometric sensor
B Interception Attack (Enrollment Phase)	2 Attack occurs before any digital processing of the biometric signal
C Adversarial Attack	3 Can be executed without access to stored templates or model internals
D Data Injection Attack	4 Requires a physical or synthesised artefact mimicking a legitimate biometric trait
E Override Feature Extractor	5 Exploits the communication channel between two pipeline components
F Feature Tampering Attack	6 Requires white-box or grey-box knowledge of the model's learned parameters
G Override Matcher	7 Attack succeeds regardless of the correctness of upstream pipeline output
H Template Attack	8 Can corrupt the enrollment database, affecting all future verifications for a victim
I Channel Intercept (Verification Phase)	9 Replaces a pipeline component entirely with a rogue implementation
J Override Decision	10 Requires privileged access to the stored template database

Table 2: Match each attack vector (A–J) with all applicable properties (1–10). Each item on the left matches **two or more** items on the right.

NAME:	ENTRY NO:
--------------	------------------

A	2,3,4	Occurs before any digital processing (2). A printed photo needs no system access (3). Requires a physical or synthetic artefact (4).
B	5,8	Exploits the channel between sensor and pre-processing (5). Corrupted enrollment data poisons all future verifications for the victim (8).
C	3,6	Can be black-box (3) or white-box/grey-box (6) — the only attack that explicitly requires gradient knowledge when white-box.
D	1,5,8	Bypasses sensor entirely (1). Injects via the channel between components (5). Poisons the enrollment database (8).
E	1,7,9	No sensor interaction needed (1). Output is attacker-controlled regardless of upstream input (7). The component itself is replaced (9).
F	1,5	No sensor interaction (1). Exploits the channel between the feature extractor and the matcher (5).
G	1,7,9	No sensor interaction (1). Score output is attacker-controlled regardless of upstream computation (7). The matcher component is replaced (9).
H	1,10	No sensor interaction required (1). Requires privileged access to the template database (10).
I	1,5	No sensor interaction (1). Exploits the channel between the database/extractor and the matcher (5).
J	1,7	No sensor interaction (1). The outcome is independent of all upstream computation, including the matcher (7).